



Budapesti Műszaki Szakképzési Centrum

PETRIK LAJOS

Két Tanítási Nyelvű Technikum

Vizsgaremek témája: időpontfoglaló rendszer

Neve: EasyArrange

Osztály: 13/E

Szakma: Szoftverfejlesztő és tesztelő technikus

Készítette: Szarvas Milán és Tóth Péter

Szakmai vizsga dokumentáció

Tartalomjegyzék

Bevezetés	6
1. A projekt célja	6
1.2 Célközönség.....	6
2. Specifikációk	7
2.1 Főbb üzleti követelmények	7
2.2 Nem funkcionális követelmények	7
3. Technológiák és függőségek	7
3.1 Backend.....	7
3.1.1 JUnit tesztek	8
3.2 Frontend	8
3.3 Adatbázis	8
4. Megvalósított funkciók.....	9
4.1 Felhasználói funkciók.....	9
4.2 Staff funkciók	9
4.3 Admin funkciók.....	9
4.4 UI/UX állapot (aktuális frontend)	10
5. Architektúra bemutatása	11
5.1 Frontend (React SPA)	11
5.2 Backend (Spring Boot REST).....	11
5.3 Database (MariaDB)	12
6. Adatáramlás	12
6.1 Az adatáramlás folyamata	12
6.1.2 Biztonság és Jogosultságkezelés.....	15
6.1.3 Hibakezelés	15
7. Üzleti logika és szabályok	15
7.1 Időpont foglalás működése	15
7.1.1 Működés áttekintése	15
7.1.2 Munkarend ellenőrzése.....	15
7.1.3 Időintervallum meghatározása	15

7.1.4 Szolgáltatás időtartamának figyelembevétele.....	15
7.1.5 Időpontok generálása	15
7.1.6 Felkészülési idő és korlátozások	15
7.1.7 Ütközések vizsgálata	15
7.1.8 Elérhető időpont feltétele	15
7.2 Munkatársi műszakok módosításának logikája	15
7.2.1 Adat-előkészítés és érvényesítés	15
7.2.2 Jogosultsági szabályok.....	16
7.2.3 Megfelelő műszak (Shift) biztosítása	16
7.2.4 Meglévő beosztás kezelése és frissítése	16
7.2.5 Új beosztás rögzítése	16
8. Telepítési és üzemeltetési útmutató.....	17
8.1. Rendszerkövetelmények (Előfeltételek)	17
8.2 GitHub repository klónozása a helyi tárhelyre	17
8.2.1 A klónozás lépései.....	17
8.3. Az adatbázis előkészítése	17
8.4. A Backend (Szerver) indítása	18
8.5. A Frontend (Kliens) indítása	18
8.6. Elérhetőségek összefoglalása	19
8.7 Teszteléshez bejelentkezés:	19
9. Fő funkciók használata	19
9.1 Regisztráció.....	19
10. API-dokumentáció	21
10.1 Alapadatok:	21
10.2 Alapfogalmak a táblázathoz	21
10.3 Felhasználók (Users)	21
10.4 Szolgáltatások (Services)	22
10.5 Munkatársak (Staff)	22
10.6 Műszakok (Shifts)	22
10.7 Munkatárs-műszak (Staff shifts)	23
10.8 Munkatárs-szolgáltatás (Staff services).....	23
10.9 Időpontfoglalás (Bookings)	24

10.10 Naptár blokkok (Calendar blocks).....	24
11. Kérés- és válaszfórmátumok	26
11.1 Jellemző kérésminták	26
11.2 Jellemző válaszbjektumok	27
11.2.1 UserResponse	27
11.2.2 ServiceResponse	28
11.2.3 StaffResponse	28
11.2.4 BookingResponse	28
11.2.5 CalendarBlockResponse	29
12. Hitelesítés és hibakezelés	30
12.1 Hitelesítés	30
12.2 Hibakezelés	30
13. ER Diagram	31
14. Adatbázis dokumentáció	31
14.1. Entity-Relationship (ER) Modell és Kapcsolatok	31
14.1.1 Főbb kapcsolati típusok:	31
14.2. Adattáblák részletes leírása	32
14.2.1. Felhasználókezelés és Szerepkörök	32
14.2.2. Szolgáltatások és Időbeosztás	32
14.2.3. Foglalási rendszer	32
14.3. Adatszótár (Fontosabb táblák szerkezete)	33
14.4. SQL Megoldások és optimalizálás	33
15. Csapatmunka	34
15.1 Verziókezelés	34
16. Fejlesztési módszertan	35
16.1 Módszertani keret	35
16.1.1 Agilis módszertan: gyors visszacsatolások és folyamatos stabilizálás	35
16.1.2 Két hetes sprintek Trello használatával	35
16.2 Tesztelési stratégia	36
16.2.1 Tesztcsomagok és lefedettség	37
16.3 Minősegbiztosítási gyakorlat	38
16.4 Build és futtatási folyamat	38

16.5 Verziókezelés és együttműködés	38
17. Jövőbeli fejlesztési lehetőségek	40
18. Projekt elérése	41
19. Források	42

Bevezetés

Az **EasyArrange** projekt megvalósítása egy izgalmas és tanulságos mérföldkő számunkra, amely a duális képzés keretein belül jöhetett létre. A képzés során kapott elméleti alapozás és a gyakorlati szemléletmód felkészített minket egy ilyen komplex rendszer fejlesztésére, de a valódi tudást a megvalósítás folyamata adta. Munkánk során rengeteg szakmai segítséget és útmutatást kaptunk tanárainktól, akiknek támogatása nélkülözhetetlen volt a felmerülő technikai kihívások leküzdésében. A fejlesztés alatt nemcsak technológiai ismereteink mélyültek el, hanem megtanultuk a rendszerszintű tervezés és a felhasználói igények összehangolásának fontosságát is.

A projekt eredményeként létrejött EasyArrange egy olyan modern időpontfoglalási és üzemeltetési rendszer, amelyet kifejezetten szépségszalonok igényeire szabtuk. Elsődleges célunk egy olyan integrált platform létrehozása volt, amely az interneten keresztül bárki számára elérhető, és egyetlen közös felületen fogja össze a szalon napi működéséhez szükséges összes munkafolyamatot.

1. A projekt célja

Az időpontfoglalási folyamat digitalizálása és egyszerűsítése mindkét fél számára – ügyfeleknek és szolgáltatóknak egyaránt – kulcsfontosságú a hatékonyság növelésében. Ennek része a telefonos kommunikáció kiváltása online foglalási, módosítási és lemondási lehetőségekkel, ami nemcsak az elmulasztott időpontok számát csökkenti, hanem a termelékenységet is növeli egy gyors, felhasználóbarát felület révén. Emellett a munkatársak számára is zökkenőmentesebb munkanapokat biztosít, mivel minden időpont egyértelműen követhető, az ütközések elkerülhetők, és a foglalások elérhetővé válnak a nap 24 órájában. Mindezt kiegészíti, hogy az összes fontos időpont és ügyféladat egy központi helyen kerül tárolásra, így könnyebbé és átláthatóbbá téve a kezelést.

1.2 Célközönség

A projektnünkben megvalósított szoftverrel több, különböző felhasználót célzunk meg.

1. Kis- és közepes méretű szépségszalonok
2. Egyéni szolgáltatók (fodrászok, manikűrösök, borbélyok, szempilla-stylistok, masszőrök)
3. A szépségiparban tevékenykedő ügyfelek, akik gyors, éjjel-nappal elérhető online időpontfoglalást igényelnek

2. Specifikációk

2.1 Főbb üzleti követelmények

A rendszer alapfunkciói közé tartozik a teljes körű ügyfélkezelés, amely a regisztrációtól és bejelentkezéstől a személyes profil menedzseléséig terjed.

Az ügyfelek a szabad időszavak függvényében foglalhatnak szolgáltatásokat a kiválasztott munkatársakhoz.

A munkatársak saját műszakrendjüket és időkizárásaikat kezelhetik, emellett az adminisztrátorokkal közösen frissíthetik a foglalások állapotát a teljes életciklus során (BOOKED, COMPLETED, NO_SHOW, CANCELLED).

Az adminisztratív jogkör kiterjed a felhasználók és szolgáltatások kezelésére is. A szoftver automatikusan kiküszöböli az ütközéseket a foglalások és naptárblokkolások között, az elérhető időpontokat pedig szigorúan a megadott műszakokhoz igazítja.

2.2 Nem funkcionális követelmények

A technológiai megvalósítás REST API alapú backendre és modern SPA frontendre épül, az adatok tárolását pedig perzisztens relációs adatbázis biztosítja. A rendszer stabilitását és biztonságát a backend oldali validációk, a DTO-k használata, valamint az egységes, globális hibakezelési logika garantálja.

3. Technológiák és függőségek

3.1 Backend

A szerveroldali logikát a **Java 21** nyelvre alapoztuk, mivel ez a legfrissebb hosszú távon támogatott verzió, amely kiváló teljesítményt és modern nyelvi elemeket kínál. Erre épül a **Spring Boot 4.0.3** keretrendszer, amely gyors és stabil alapot biztosít a vállalati alkalmazások fejlesztéséhez.

Az adatkezelésért a **spring-boot-starter-data-jpa** felel, amely jelentősen leegyszerűsíti az adatbázis-műveleteket az objektum-relációs leképzés (ORM) segítségével.

A rendszer biztonságát és a jogosultságkezelést a **spring-boot-starter-security** modul garantálja, míg a **spring-boot-starter-webmvc** a RESTful végpontok kialakításáért és a HTTP kérések kiszolgálásáért felel.

Az adatok integritását a **spring-boot-starter-validation** biztosítja a bejövő kérések ellenőrzésével, a **MariaDB JDBC driver** pedig a stabil adatbázis-kapcsolatot létesíti.

Végül a **Jackson** könyvtárat használjuk, amely az adatok automatikus JSON formátumba való alakításáért (szerializációjáért) felel a kliens és a szerver között.

3.1.1 JUnit tesztek

A projekt tesztelési alapjait a **spring-boot-starter-test** könyvtár biztosítja, amely egy átfogó gyűjtőcsomagként tartalmazza a legfontosabb tesztelési eszközöket, többek között a JUnit 5 keretrendszert, az AssertJ állításokat és a Spring kontextus teszteléséhez szükséges segédfüggvényeket.

Ezt egészíti ki a **mockito-core**, amely lehetővé teszi a függőségek (példából fakadóan szervizek vagy adatbázis-kapcsolatok) helyettesítését úgynevezett "mock" objektumokkal, így az egyes komponensek izoláltan, a külvilágtól függetlenül vizsgálhatók.

A folyamat integrációját a **mockito-junit-jupiter** modul teszi teljessé, amely hidat képez a két rendszer között: ez a kiterjesztés felel azért, hogy a Mockito annotációi (mint például a `@Mock` vagy `@InjectMocks`) zökkenőmentesen együttműködjenek a JUnit 5 életciklusával, automatizálva a tesztadatok előkészítését és a mock objektumok inicializálását.

3.2 Frontend

A felhasználói felület a **React 19** könyvtárral készül, amely komponensalapú felépítésével gyors és skálázható UI fejlesztést tesz lehetővé. Az alkalmazás alapjául a **Vite 8**-at választottuk, mivel ez a legmodernebb és leggyorsabb build eszköz, amely villámgyors fejlesztői élményt nyújt.

Az oldalak közötti navigációt a **React Router DOM 7** kezeli, biztosítva a zökkenőmentes Single Page Application (SPA) működést.

A szerverrel való kommunikációt az **Axios** végzi, míg az adatok lekérését, gyorsítótárazását és szinkronizálását a **@tanstack/react-query** menedzseli a hatékonyság érdekében.

A modern vizuális élményért és a gördülékeny animációkért a **Framer Motion** felel. A kód minőségének és egységességének fenntartását az **ESLint** biztosítja, amely már a fejlesztés során jelzi a potenciális hibákat.

3.3 Adatbázis

A rendszer adattárolási rétege egy **MariaDB / MySQL kompatibilis SQL séma**, amelyet a megbízhatósága és a relációs adatkezelésben nyújtott kiváló teljesítménye miatt választottunk. Ez a technológia tökéletesen alkalmas az összetett foglalási logikák, ütközések és felhasználói adatok strukturált, biztonságos tárolására.

4. Megvalósított funkciók

Az alkalmazás alapvetően három különböző szerepkört különböztet meg: az **Ügyfeleket**, akik a szolgáltatásokat igénybe veszik; a **Staff (személyzet)** tagokat, akik a tényleges munkát végzik és a saját idejüket menedzselik; valamint az **Adminisztrátorokat**, akik a teljes rendszer felett rendelkeznek.

4.1 Felhasználói funkciók

Az **Ügyfelek** számára a rendszer egy intuitív foglalási felületet biztosít, amely a regisztrációval és a biztonságos bejelentkezéssel veszi kezdetét. Saját profiljukban bármikor módosíthatják adataikat vagy kezdeményezhetnek jelszócserét a fiókjuk biztonsága érdekében. A foglalási folyamat során a felhasználók kiválaszthatják a kívánt szolgáltatást és a hozzá tartozó munkatársat, majd a naptárban ellenőrizhetik a szabad időszavakat. A rendszer lehetővé teszi a szolgáltatók elérhetőségének szűrését is, így az ügyfél láthatja, hogy az adott napon ki és meddig érhető el a műszakbeosztása alapján. Végezetül a "Saját foglalások" menüpontban nyomon követhetik minden korábbi és jövőbeli időpontjukat.

4.2 Staff funkciók

A **Staff (személyzet)** tagjai a saját operatív munkájukat támogató felülethez férnek hozzá, ahol elsődleges feladatuk a saját műszakbeosztásuk menedzselése. Itt láthatják és kezelhetik az ügyfelek által hozzájuk beérkező foglalásokat, valamint joguk van a foglalások státuszának frissítésére (például ha a vendég megérkezett vagy éppen lemondta az időpontot). A munkavállalók emellett úgynevezett "Time off" blokkokat hozhatnak létre vagy törölhetnek; ez a funkció szolgál a szünetek vagy egyéb akadályoztatások rögzítésére, megakadályozva, hogy ezekre az időszakokra bárki időpontot foglalhasson.

4.3 Admin funkciók

Az **Adminisztrátorok** felelnek a rendszer üzleti logikájának fenntartásáért és a felhasználók felügyeletéért. Feladatuk a kínált szolgáltatások palettájának kezelése, ami magában foglalja az új szolgáltatások létrehozását és a meglévők szerkesztését vagy törlését. Ők végzik a munkatársak és szolgáltatások összerendelését, meghatározva, hogy melyik dolgozó milyen típusú feladatot láthat el. A felhasználókezelő felületen keresztül (Manage Users) az adminok módosíthatják a regisztráltak szerepköreit vagy szükség esetén törölhetik a profilokat, a "My Staff" nézetben pedig globálisan ellenőrizhetik az összes munkavállaló műszakrendjét.

4.4 UI/UX állapot (aktuális frontend)

A felhasználói élményt egy modern, szerepkör-alapú navigációs rendszer alapozza meg, amely a felső menüsávban csak azokat az opciókat jeleníti meg, amelyekre az adott felhasználónak jogosultsága van. A felület teljes mértékben reszponzív, így mobil- és desktop eszközökön egyaránt kényelmesen használható. A rendszer egy globális értesítési komponenst használ a visszajelzésekre (például sikeres mentés vagy hibaüzenet esetén), a szerkesztőfelületek pedig stíluszott formkontrollokkal teszik egyszerűvé az adatbevitelt. A biztonság érdekében a védett oldalakhoz való hozzáférést szigorú szerepkör-ellenőrzés korlátozza, az esetlegesen elépelt URL-címek vagy nem létező oldalak esetén pedig egy egyedi 404 Not Found oldal tájékoztatja a felhasználót.

5. Architektúra bemutatása

A háromréteges elkülönítés azért előnyös, mert a rétegek egymástól függetlenül fejleszthetők és karbantarthatók: ha például az adatbázist le kell cserélni, az nem igényel módosítást a felhasználói felületen, vagy fordítva. Emellett a szétválasztás növeli a biztonságot és a rendszer skálázhatóságát is.

5.1 Frontend (React SPA)

A frontend réteg felelős a felhasználókkal való közvetlen kapcsolattartásért; ez az a felület, ahol az ügyfelek és a munkatársak interakcióba lépnek a rendszerrel. A megvalósításhoz egy **Single Page Application (SPA)** modellt választottunk, ami azt jelenti, hogy az oldal nem frissül újra minden kattintásnál, így alkalmazásszerű élményt nyújt.

Az alkalmazás logikai vázát a kiterjedt **útvonalválasztó (route) rendszer** adja, amely külön kezeli a publikus oldalakat (mint a kezdőlap, szolgáltatások vagy regisztráció), a személyes profilkezelést, valamint a dedikált munkatársi és adminisztrátori felületeket. A technikai háttérrel az **Axios** kliens biztosítja az API-kommunikációhoz, a szerverről érkező adatok hatékony kezeléséért pedig a **React Query** felel. A bejelentkezett felhasználó adatait és jogosultságait egy központi **AuthContext** tárolja, így a rendszer minden pillanatban tudja, melyik menüpontot jelenítheti meg az adott személynek.

5.2 Backend (Spring Boot REST)

A backend réteg a rendszer „agya”, ahol az érdemi feldolgozás történik. Feladata a kliens felől érkező kérések fogadása, validálása és az üzleti szabályok érvényesítése. A felépítése több belső szintre oszlik:

- **Controller réteg:** Ez a beviteli kapu, itt érkeznek be a REST kérések a frontend felől.
- **Service réteg:** Itt lakik az alkalmazás „lelke”, az üzleti logika. Ez a réteg számolja ki az elérhető szabad időpontokat, ellenőrzi az ütközéseket a naptárban, és kezeli a foglalások státuszát.
- **Repository réteg:** A Spring Data JPA segítségével közvetlen kapcsolatot tart az adatbázissal, elvégezve a szükséges lekérdezéseket.
- **DTO és Mapper réteg:** Fontos biztonsági és tervezési elem, amely elválasztja a belső adatbázis-struktúrát a külvilágnak küldött adatoktól, így csak a szükséges információk hagyják el a szervert.
- **GlobalExceptionHandler:** Gondoskodik arról, hogy bármilyen hiba esetén a frontend egy érthető és egységes hibaüzenetet kapjon vissza.

5.3 Database (MariaDB)

Az adatbázis réteg felelős az információk biztonságos és strukturált tárolásáért. Egy **MariaDB** alapú relációs sémát használunk, amely garantálja az adatok közötti összefüggéseket (például egy foglалás nem létezhet létező ügyfél vagy munkatárs nélkül).

A tervezés során nagy hangsúlyt fektettünk az idegen kulcsokra (FK) és az egyedi korlátozásokra (Unique constraints), hogy megakadályozzuk az adathibákat. A rendszer rugalmasságát speciális **kapcsolótáblák** biztosítják: ezek segítségével kezeljük a munkatársak és a hozzájuk rendelt szolgáltatások (staff-service), valamint a munkatársak és a munkaidejük (staff-shift) közötti összetett kapcsolatokat.

6. Adatáramlás

6.1 Az adatáramlás folyamata

UI → Frontend (Axios + React Query) → JSON Response → Controller → DTO/Mapper → Service Layer → JPA Repository → Hibernate Entity → Database

1. A frontend Axios segítségével hívja meg az API-t, React Query kezeli az állapotot, majd a komponensek megjelenítik az adatokat a felhasználói felületen.

```
Frontend > src > api > JS userApi.js > ...
1  import Axios from 'axios';
2  import { BASE_URL, getAuthHeaders, request } from './apiClient';
3
4  export const userApi = {
5    async getUsers() {
6      const response = await Axios.get(`${BASE_URL}/users`);
7      return response.data;
8    },
9
10   async getUser(userId) {
11     return request(() => Axios.get(`${BASE_URL}/users/${userId}`));
12   },
13 }
```

1.1 Kép: Axios get metódus használata.

```
Frontend > src > hooks > queries > JS useUsers.js > useUsers > queryFn
1  import { useQuery } from '@tanstack/react-query'
2  import { userApi } from '../api/userApi'
3
4  export function useUsers() {
5    return useQuery({
6      queryKey: ['users'],
7      queryFn: userApi.getUsers
8    })
9  }
10
11 export default useUsers
```

1.2 Kép: React Query állapotkezelése.

```
20  const { data: users = [], isLoading, error } = useUsers()
```

1.3 Kép: A komponensek így érik el.

2. A Controller REST végpontokon keresztül JSON formátumban adja vissza az adatokat.

```
31  @GetMapping()
32  public List<UserResponse> getUsers() {
33    //get current authenticated username (typically email)
34    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
35    String currentEmail = auth != null ? auth.getName() : null;
36
37    //retrieve all users and filter out the currently logged-in user (by email)
38    return userService.findAll().stream()
39      .filter( UserResponse u -> currentEmail == null || !currentEmail.equals(u.email()))
40      .collect(Collectors.toList());
41  }
```

1.4 Kép: Controller réteg UserResponse DTO-val tér vissza.

3. A DTO és Mapper réteg biztosítja, hogy csak szükséges adatok kerüljenek ki a frontend felé.

```
5  public record UserResponse(
6    Long userId,
7    String name,
8    String email,
9    String phoneNumber,
10   String profilePicture,
11   RoleResponse role
12 ) {
```

1.5 Kép: UserResponse DTO nem adja vissza a jelszót biztonsági okokból.

4. A Service réteg tartalmazza az üzleti logikát (pl. validációk, ütközésvizsgálat).

```
41  
42     public UserResponse findUserId(Long id) { 3 usages  @ peter.toth  
43         Optional<User> user = userRepository.findById(id);  
44         return user.map(userResponseMapper).orElseThrow(() -> new IllegalArgumentException("User not found!"));  
45     }  
46
```

1.6 Kép: Exception küldés, ha a felhasználó nem található a megadott Id-val.

5. A JPA Repository réteg végzi az adatelérést.

```
@Repository 8 usages  @ peter.toth  
public interface UserRepository extends JpaRepository<User, Long> {  
    Optional<User> findUserByEmail(String email); 1 usage  @ peter.toth  
}
```

1.7 Kép: JPA Repository megszerzi az adatbázisból a felhasználót email alapján.

6. Az adatbázisban tárolt adatokat a Hibernate entity objektumokká alakítja.

```
@Entity  @ Szarvas Milán +1  
@Table(name = "user", schema = "easyarrange", uniqueConstraints = {@UniqueConstraint(name = "email",  
    columnNames = {"email"})})  
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name = "user_id", nullable = false)  
    private Long id;  
  
    @Column(name = "name", nullable = false, length = 50) 2 usages  
    @Size(max = 50, message = "Name must not exceed 50 characters")  
    private String name;  
  
    @Column(name = "email", nullable = false) 2 usages  
    @Size(max = 255, message = "Email must not exceed 255 characters")  
    private String email;  
  
    @Column(name = "profile_picture") 2 usages  
    private String profilePicture;  
  
    @Column(name = "password", nullable = false) 2 usages  
    private String password;  
  
    @ManyToOne(fetch = FetchType.LAZY, optional = false) 2 usages  
    @JoinColumn(name = "role_id", nullable = false)  
    @OnDelete(action = OnDeleteAction.RESTRICT)  
    private Role role;  
  
    @OneToMany(mappedBy = "user") 2 usages  
    private List<Booking> bookings = new ArrayList<>();  
  
    @OneToOne(mappedBy = "user") 2 usages  
    private Staff staff;  
  
    @Size(max = 12) 2 usages  
    @Column(name = "phone_number", nullable = false, length = 12)  
    private String phoneNumber;
```

1.8 Kép: A user adatbázis táblából Hibernate entity készült.

6.1.2 Biztonság és Jogosultságkezelés

A rendszer védelmét és a szerepkör alapú hozzáférést a **Spring Security** biztosítja. A biztonsági architektúra alapjai (SecurityConfig, szerepkörök) már rendelkezésre állnak, azonban a rendszer teljes körű működéséhez a **JWT-alapú hitelesítés** befejezése még szükséges.

6.1.3 Hibakezelés

A backend egységes válaszairól a **GlobalExceptionHandler** gondoskodik. Ez a központosított megoldás:

- **Kiszámíthatóságot biztosít:** A frontend mindig szabványos formátumú hibaobjektumot és releváns HTTP státuszkódot (pl. 400, 403, 404, 500) kap.
- **Tisztább kódot eredményez:** Megszünteti a redundanciát, mivel a validációs, üzleti és szerveroldali hibák kezelése egy helyen összpontosul.

7. Üzleti logika és szabályok

Az üzleti logika (business logic) azok a szabályok, folyamatok és döntések, amelyek meghatározzák, hogyan működik egy alkalmazás a felhasználói igények és a vállalati célok alapján. Ez magában foglalja például a validációkat, tranzakciókat, adatfeldolgozást és a rendszer működésének egyéb lényegi részeit.

Ebben a fejezetben bemutatjuk az alkalmazás legfontosabb funkcióinak a működését.

7.1 Időpont foglalás működése

A weboldalunk egyik fő pillére az időpont foglalás, anélkül maga a koncepció sem születhetett volna meg. Ha például fodráshoz szeretnénk menni és szeretnénk időpontot kérni könnyebben meg tudjuk ezt tenni online. Fontos szemléltetni, hogy mi alapján működik az időpont foglalása, hogy megvizsgáljuk a helyes működését.

7.1.1 Működés áttekintése

Az elérhető időpontok generálása a következő bemeneti paraméterek alapján történik:

Azaz a kiválasztott munkatárs (staffId), a kiválasztott dátum (selectedDate) és a kiválasztott szolgáltatás (ServiceId).

A rendszer ezek alapján egy listát ad vissza a foglalható időintervallumokról.

7.1.2 Munkarend ellenőrzése

Első lépésben a rendszer lekéri a kiválasztott munkatárs műszakjait, majd kiválasztja azt, amelyik megfelel a kiválasztott napnak. Ha az adott napra **nincs műszak**, akkor **nem generálható időpont**, a rendszer üres listát ad vissza.

Ez biztosítja, hogy csak ténylegesen dolgozó időszakokra lehessen foglalni.

7.1.3 Időintervallum meghatározása

A kiválasztott naphoz tartozó műszak alapján meghatározásra kerül a **nap kezdete** (startOfDay) és a **nap vége** (endOfDay).

Ez lesz az az intervallum, amelyen belül a rendszer időpontokat generál.

7.1.4 Szolgáltatás időtartamának figyelembevétele

A rendszer az adatbázisból lekéri a szolgáltatás hosszát, és csak akkor engedélyezi a foglalást, ha az **teljes egészében belefér** a munkatárs műszakidejébe.

7.1.5 Időpontok generálása

A szoftver a műszak kezdetétől **15 perces** lépésközzel haladva vizsgálja az idővonalat. Minden lépésnél kiszámítja a befejezési időt és ellenőrzi a **műszakhatárokat**, így biztosítva az egységes és optimalizált időrácsot.

7.1.6 Felkészülési idő és korlátozások

A felkészülési idő és az operatív biztonság érdekében a rendszer csak a jelen időpontnál legalább 3 órával későbbi kezdéseket jeleníti meg, korlátozva a váratlan last-minute foglalásokat.

7.1.7 Ütközések vizsgálata

Minden időpontnál kettős ellenőrzés fut le: a rendszer szűri a más ügyfelekkel való időbeli átfedéseket, valamint figyelembe veszi a manuálisan rögzített naptárblokkolásokat, például a szüneteket vagy szabadságokat.

7.1.8 Elérhető időpont feltétele

- csak műszakidőn belül lehet foglalni
- a szolgáltatás teljes időtartamának bele kell férnie
- 15 perces időrács használata
- minimum 3 órás előfoglalási idő
- nincs átfedés más foglalással
- nincs átfedés blokkolt időszakkal

7.2 Munkatársi műszakok módosításának logikája

Ez a folyamat felel azért, hogy egy adott munkatárs munkaidejét egy konkrét napra vonatkozóan frissítsük vagy létrehozzuk. A metódus tranzakcionális, így garantálja, hogy az adatbázis-műveletek csak akkor véglegesednek, ha minden lépés hiba nélkül lefutott.

7.2.1 Adat-előkészítés és érvényesítés

A folyamat a beérkező kérés feldolgozásával indul, ahol a rendszer kinyeri a munkatárs azonosítóját, a napot és a munkaidő kereteit. Első lépésben egy alapvető időpont-ellenőrzés történik, amely biztosítja, hogy a kezdési időpont megelőzze a befejezést. Ezt követi a munkatárs létezésének ellenőrzése az adatbázisban; ha az illető nem található, a folyamat hibaüzenettel leáll.

7.2.2 Jogosultsági szabályok

A rendszer ellenőrzi a munkatárshoz tartozó felhasználói szerepkört is. Csak meghatározott szerepkörrel rendelkező felhasználók számára kezelhetők a műszakok, így megakadályozható, hogy olyan személyekhez rendeljünk munkaidőt, akiknek a feladatköre ezt nem indokolja.

7.2.3 Megfelelő műszak (Shift) biztosítása

Ahelyett, hogy minden alkalommal új bejegyzést hozna létre, a rendszer megvizsgálja, létezik-e már az adatbázisban az adott napra és időintervallumra vonatkozó általános műszak-definíció. Ha nincs ilyen, létrehoz egyet és elmenti, ha viszont létezik, akkor azt fogja felhasználni a hozzárendeléshez. Ezzel elkerülhető az adatok felesleges többszörözése.

7.2.4 Meglévő beosztás kezelése és frissítése

A rendszer lekérdezi, hogy a munkatársnak van-e már rögzített beosztása az adott napra:

- Ha a munkatárs már pontosan ebben a műszakban van, a folyamat nem végez módosítást, csupán visszaigazolja a meglévő állapotot.
- Ha a munkatársnak van beosztása, de más időpontban, a rendszer törli a régi hozzárendelést, hogy helyet adjon az újnak.

7.2.5 Új beosztás rögzítése

A folyamat zárásaként a rendszer összekapcsolja a munkatársat a kiválasztott vagy frissen létrehozott műszakkal. Az új hozzárendelés mentése után a rendszer egy válaszbjektumot készít, amely tartalmazza a frissített beosztás minden lényeges adatát.

8. Telepítési és üzemeltetési útmutató

Ez a fejezet az **EasyArrange** alkalmazás helyi környezetben történő telepítéséhez és futtatásához szükséges lépéseket ismerteti.

8.1. Rendszerkövetelmények (Előfeltételek)

A futtatáshoz az alábbi szoftverek megléte szükséges a gazdagépen:

- **Java SDK 17** vagy újabb (a Spring Boot backendhez).
- **Node.js** (LTS verzió ajánlott) és **npm** csomagkezelő.
- **MySQL Server 8.0** vagy **MariaDB**.
- **Maven** (a projektben található mvnw wrapper miatt nem kötelező globálisan telepíteni).

8.2 GitHub repository klónozása a helyi tárhelyre

A telepítés első lépése a projekt forráskódjának letöltése a GitHub szervereiről a saját számítógépére. A klónozással nemcsak a fájlokat kapja meg, hanem a projekt teljes verziótörténetét is.

8.2.1 A klónozás lépései

A művelet végrehajtásához nyisson meg egy terminált (parancssort vagy Git Bash-t) abban a mappában, ahová a projektet menteni szeretné, majd futtassa az alábbi parancsokat:

1. `git clone https://github.com/Twixy01/EasyArrange.git`
2. `cd EasyArrange`

8.3. Az adatbázis előkészítése

Az alkalmazás indítása előtt létre kell hozni az adatbázis-sémát és fel kell tölteni a kezdeti adatokkal.

1. Indítsa el a MySQL szerveret.
2. Hozzon létre egy `easyarrange` nevű adatbázist és állítsuk be az illesztést:

```
CREATE DATABASE easyarrange  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_hungarian_ci;
```

3. Importálja az easyarrange.sql fájlt az adatbázisba (Database mappában találja).
4. **Fontos:** Ellenőrizze a Backend/src/main/resources/application.properties fájlban, hogy az adatbázis felhasználóneve és jelszava egyezik-e a helyi beállításokkal!

```
1      spring.application.name=Backend
2
3      spring.datasource.url=jdbc:mariadb://localhost:3306/easyarrange
4      spring.datasource.username=root
5      spring.datasource.password=
```

1.9 Kép: MariaDB adatbázishoz csatlakozás Spring Boot-tal.

8.4. A Backend (Szerver) indítása

A backend szolgáltatás felel az üzleti logikáért és az adatbázis-kapcsolatért.

1. Nyisson egy terminált a projekt **Backend** mappájában.
2. Futtassa az alkalmazást az alábbi paranccsal:
 - **Windows:** .\mvnw.cmd spring-boot:run
 - **Linux/macOS:** ./mvnw spring-boot:run
3. A szerver alapértelmezetten a http://localhost:8080 címen lesz elérhető.

8.5. A Frontend (Kliens) indítása

A felhasználói felület React alapon, Vite keretrendszerrel készült.

1. Nyisson egy új terminált a projekt **Frontend** mappájában.
2. Telepítse a szükséges függőségeket (külső könyvtárakat):

```
npm install
```

(Megjegyzés: A fejlesztés során használt modulok, mint a framer-motion, axios és react-router-dom automatikusan telepítésre kerülnek a package.json alapján.)

3. Indítsa el a fejlesztői szerveret:

```
npm run dev
```

4. A kliens alapértelmezetten a http://localhost:5173 címen indul el.

8.6. Elérhetőségek összefoglalása

Komponens	URL / Port	Leírás
Frontend	http://localhost:5173	A webalkalmazás felhasználói felülete.
Backend API	http://localhost:8080	Az API végpontok és az adatbázis kapcsolata.
Adatbázis	localhost:3306	MySQL / MariaDB kiszolgáló.

8.7 Teszteléshez bejelentkezés:

ADMIN (aki STAFF is): *email:* peter@gmail.com , *jelszó:* 1234

STAFF: *email:* milan@gmail.com, *jelszó:* 1234

CUSTOMER: *email:* bence@gmail.com, *jelszó:*1234

9. Fő funkciók használata

Ebben a fejezetben a rendszer fő funkcióinak használata kerül bemutatásra, a felhasználói szerepkörök szerinti bontásban. A látogatók a regisztráció során megadhatják alapadataikat, majd bejelentkezést követően hozzáférnek a foglalási felülethez, ahol kiválaszthatják a kívánt szolgáltatást, munkatársat, időpontot és véglegesíthetik foglalásukat. A felhasználói profil oldalon lehetőség van a személyes adatok megtekintésére, a korábbi és aktuális foglalások kezelésére, valamint a jelszó módosítására. A rendszer külön funkciókat biztosít a munkatársak számára is, akik kezelhetik saját műszakbeosztásukat, megadhatják kieső időpontjaikat, valamint nyomon követhetik és frissíthetik a hozzájuk tartozó foglalások állapotát. Az adminisztrátori felületen a teljes rendszer feletti irányítás valósul meg, beleértve a felhasználók és szolgáltatások kezelését, valamint a munkavállalók beosztásának és elérhetőségének felügyeletét.

9.1 Regisztráció

A rendszer használatához a felhasználók első lépésként a /register oldalon tudnak regisztrálni, ahol meg kell adniuk a nevüket, e-mail címüket, telefonszámukat, valamint egy jelszót. A regisztrációt követően a /login oldalon van lehetőség bejelentkezni az e-mail cím és a jelszó megadásával.

Bejelentkezés után a felhasználók a **/booking** oldalon tudnak időpontot foglalni. A foglalási folyamat során először ki kell választani a kívánt szolgáltatást, majd az adott szolgáltatáshoz tartozó munkatársat. Ezt követően a felhasználó kiválasztja a megfelelő dátumot, majd az adott napon elérhető időszavok közül választ. A folyamat végén a foglalást meg kell erősíteni.

A felhasználói adatok és a korábbi foglalások a **/profile** oldalon érhetők el, ahol a felhasználó áttekintheti és kezelheti saját adatait. A jelszó módosítására külön felületen, a **/profile/change-password** oldalon van lehetőség.

A munkatársak számára külön funkciók állnak rendelkezésre. A **/staff/shifts** oldalon kezelhetik a műszakjaikat, a **/staff/time-off** oldalon beállíthatják azokat az időszakokat, amikor nem elérhetők, míg a **/staff/bookings** oldalon a hozzájuk tartozó foglalások állapotát kezelhetik.

Az adminisztrátorok számára a rendszer további kezelőfelületeket biztosít. A **/admin/users** oldalon a felhasználók kezelése történik, a **/admin/services** oldalon a szolgáltatások konfigurálhatók, míg a **/admin/my-staff** oldalon a munkavállalók beosztásai és elérhetőségei követhetők nyomon.

10. API-dokumentáció

Ez a fejezet a rendszer alkalmazásprogramozási interfészének (API) részleteit tartalmazza. Az API teszi lehetővé a kliensoldal (frontend) és a szerveroldal (backend) közötti kommunikációt. A backend szolgáltatások eléréséhez szabványos HTTP-kéréseket használunk.

10.1 Alapadatok:

- **Végpontok előtagja (Prefix):** /api
- **Adatformátum:** JSON
- **Tartalomtípus:** application/json

10.2 Alapfogalmak a táblázathoz

A dokumentáció megértéséhez az alábbi fogalmakat érdemes tisztázni:

Metódus (HTTP Method): Meghatározza a kérés típusát (pl. GET az adatok lekéréséhez, POST új adat létrehozásához, PUT módosításhoz, DELETE törléshez).

Végpont (Endpoint): Az az egyedi URL útvonal, ahol az adott erőforrás elérhető. A kapcsos zárójelben lévő részek (pl. {userId}) változókat jelölnek, ide az adott elem egyedi azonosítóját kell behelyettesíteni.

Controller metódus: A szerveroldali kódban található konkrét függvény neve, amely lefut, amikor a kérés beérkezik az adott végpontra. Ez felelős az üzleti logika végrehajtásáért.

10.3 Felhasználók (Users)

Metódus	Végpont	Controller metódus
GET	/api/users	getUsers()
GET	/api/users/{userId}	getUser()
POST	/api/users/auth/login	findUserForLogin()
POST	/api/users/register	addUser()
PUT	/api/users/{userId}	updateUser()
PUT	/api/users/admin/{userId}	updateUserAsAdmin()
DELETE	/api/users/{userId}	deleteUser()

10.4 Szolgáltatások (Services)

Metódus	Végpont	Controller metódus
GET	/api/services	getServices()
GET	/api/services/ordered	getServicesOrderedByName()
GET	/api/services/{serviceId}	getServiceById()
GET	/api/services/name/{serviceName}	getServiceByName()
POST	/api/services/create	createService()
PUT	/api/services/{serviceId}	updateService()
DELETE	/api/services/{serviceId}	deleteService()

10.5 Munkatársak (Staff)

Metódus	Végpont	Controller metódus
GET	/api/staff	getStaff()
GET	/api/staff/{staffId}	getStaffById()
GET	/api/staff/user/{userId}	getStaffByUserId()
POST	/api/staff/register	addStaff()
DELETE	/api/staff/{staffId}	deleteStaff()

10.6 Műszakok (Shifts)

Metódus	Végpont	Controller metódus
GET	/api/shifts	getAll()
GET	/api/shifts/{id}	getById()
GET	/api/shifts/start?time='HH:mm'	getByStart()
GET	/api/shifts/end?time='HH:mm'	getByEnd()
POST	/api/shifts/create	create()

PUT	/api/shifts/{id}	update()
DELETE	/api/shifts/{id}	delete()

10.7 Munkatárs-műszak (Staff shifts)

Metódus	Végpont	Controller metódus
GET	/api/staff-shifts	getAllStaffShifts()
GET	/api/staff-shifts/{staffId}/{shiftId}	getStaffShiftById()
GET	/api/staff-shifts/staff/{staffId}	getShiftsByStaffId()
GET	/api/staff-shifts/shift/{shiftId}	getStaffByShiftId()
GET	/api/staff-shifts/staff/{staffId}/between?start='HH:mm&end=HH:mm'	getAllShiftsByStaffIdBetweenShifts()
POST	/api/staff-shifts/update	updateStaffShift()
POST	/api/staff-shifts/create	createStaffShift()
DELETE	/api/staff-shifts/{staffId}/{shiftId}	deleteStaffShift()

10.8 Munkatárs-szolgáltatás (Staff services)

Metódus	Végpont	Controller metódus
GET	/api/staff-services	getAllStaffServices()
GET	/api/staff-services/{staffId}/{serviceId}	getStaffServiceById()
GET	/api/staff-services/staff/{staffId}	getServicesByStaffId()
GET	/api/staff-services/service/{serviceId}	getStaffByServiceId()
POST	/api/staff-services/create	createStaffService()
DELETE	/api/staff-services/{staffId}/{serviceId}	deleteStaffService()

10.9 Időpontfoglalás (Bookings)

Metódus	Végpont	Controller metódus
GET	/api/bookings	getAll()
GET	/api/bookings/{id}	getById()
GET	/api/bookings/staff/{staffId}	getByStaff()
GET	/api/bookings/user/{userId}	getUser()
GET	/api/bookings/between?start='...&end=...'	getBetween()
GET	/api/bookings/staff/{staffId}/available-slots?selectedDate='YYYY-MM-DD&serviceId={id}'	getAvailableSlots()
GET	/api/bookings/status?status='BOOKED CANCELLED COMPLETED NO_SHOW'	getStatus()
GET	/api/bookings/staff/{staffId}/overlaps?start='...&end=...'	getOverlaps()
POST	/api/bookings/create	create()
PUT	/api/bookings/{id}/{isStaff}	update()
POST	/api/bookings/cancel/{id}	cancel()
DELETE	/api/bookings/{id}/hard	hardDelete()

10.10 Naptár blokkok (Calendar blocks)

Metódus	Végpont	Controller metódus
GET	/api/calendar-blocks	getCalendarBlocks()
GET	/api/calendar-blocks/{calendarBlockId}	getCalendarBlockById()
GET	/api/calendar-blocks/staff/{staffId}	getCalendarBlocksByStaffId()
GET	/api/calendar-blocks/between?start=...&end=...	getCalendarBlocksBetween()

GET	/api/calendar-blocks/staff/{staffId}/between?start=...&end=...	getCalendarBlocksByStaffBetween()
POST	/api/calendar-blocks/create	createCalendarBlock()
PUT	/api/calendar-blocks/{calendarBlockId}	updateCalendarBlock()
DELETE	/api/calendar-blocks/{calendarBlockId}	deleteCalendarBlock()

11. Kérés- és válaszformátumok

11.1 Jellemző kéreisminták

POST /api/users/register

```
{  
  "name": "Teszt Elek",  
  "email": "teszt@example.com",  
  "phoneNumber": "+36301234567",  
  "password": "titok123",  
  "role": { "roleId": 2, "name": "CUSTOMER" }  
}
```

POST /api/users/auth/login

```
{  
  "email": "teszt@example.com",  
  "password": "titok123"  
}
```

POST /api/bookings/create

```
{  
  "staffId": 2,  
  "userId": 5,  
  "startDateTime": "2026-05-10T10:00",  
  "endDateTime": "2026-05-10T10:40",  
  "serviceId": 1  
}
```

PUT /api/bookings/{id}/{isStaff}

```
{  
  "startDateTime": "2026-05-10T10:00",  
  "endDateTime": "2026-05-10T10:40",  
  "serviceId": 1,  
  "status": "COMPLETED"  
}
```

POST /api/calendar-blocks/create

```
{  
  "title": "Time off",  
  "startDateTime": "2026-05-11T00:00",  
  "endDateTime": "2026-05-12T00:00",  
  "staffId": 3  
}
```

11.2 Jellemző válaszobjektumok

A válaszobjektumokat recordként mutatjuk be. Azért használtunk recordot, mert az automatikusan megcsinálja a háttérben a konstruktort és a getter metódusokat.

11.2.1 UserResponse

```
public record UserResponse(  
    Long userId,  
    String name,  
    String email,  
    String phoneNumber,  
    String profilePicture,  
    RoleResponse role  
)
```

```
{  
}
```

11.2.2 ServiceResponse

```
public record ServiceResponse(  
    Long serviceId,  
    String name,  
    Integer price,  
    Integer duration,  
    String description,  
    String image  
) {  
}
```

11.2.3 StaffResponse

```
public record StaffResponse(  
    Long staffId,  
    UserResponse user,  
    String title,  
    String bio,  
    List<ServiceResponse> services  
) {  
}
```

11.2.4 BookingResponse

```
public record BookingResponse(  

```

```

    Long bookingId,

    StaffResponse staff,

    UserResponse user,

    @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm")
    LocalDateTime startDateTime,

    @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm")
    LocalDateTime endDateTime,

    ServiceResponse service,

    String status

) {
}

```

11.2.5 CalendarBlockResponse

```

public record CalendarBlockResponse(

    Long calendarBlockId,

    String title,

    @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm")
    LocalDateTime startDateTime,

    @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm")
    LocalDateTime endDateTime,

    StaffResponse staff

) {
}

```

12. Hitelesítés és hibakezelés

12.1 Hitelesítés

A backenden HTTP Basic auth van bekapcsolva, viszont a SecurityConfig jelenleg minden endpointot **permitAll** szintű hozzáférésre engedélyez. Ez azt jelenti, hogy a jogosultságellenőrzés jelenleg inkább a **frontend UX-korlátozásaira épül**, mivel a backend oldali validáció — például a Bearer token ellenőrzése — nem teljesen van kikényszerítve.

A frontend ugyan tárolja a user- és tokenadatokat a localStorage-ben, de a token érvényességének és jogosultságellenőrzésének backend oldali validációja jelenleg nem teljes körű.

12.2 Hibakezelés

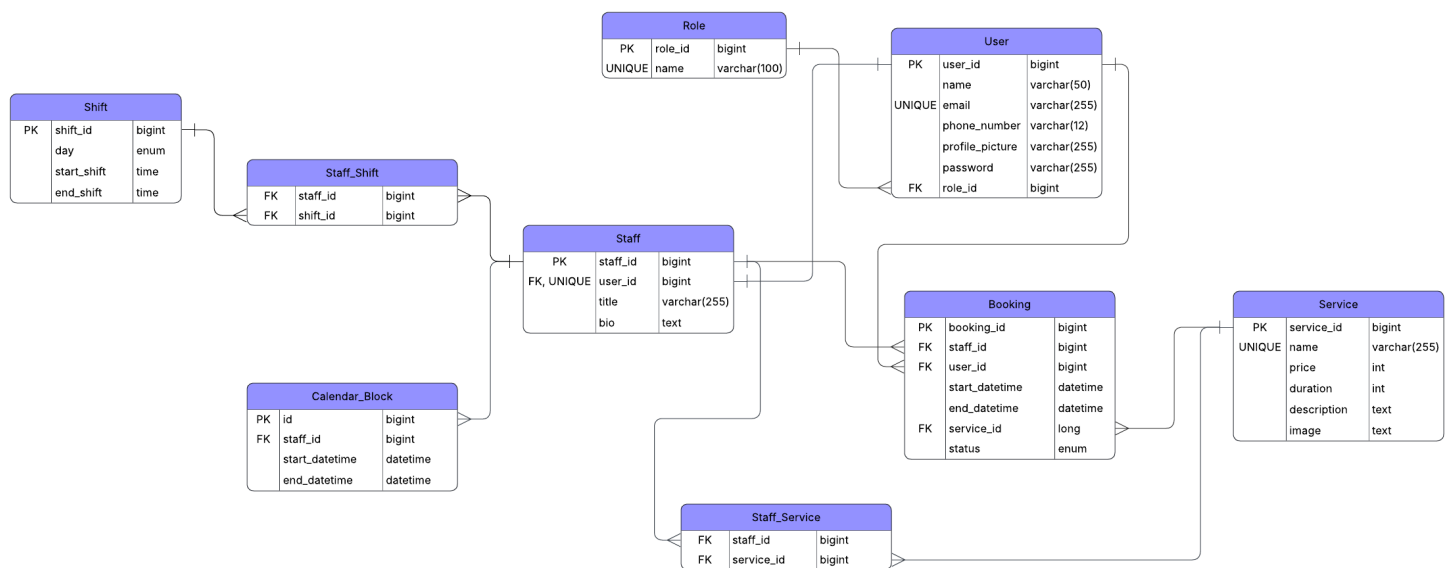
Az alkalmazás központosított hibakezelését a **GlobalExceptionHandler** látja el, amely egységes, **ProblemDetail** specifikáció alapú válaszobjektumokat küld a kliensnek. Ez biztosítja, hogy a frontend minden esetben szabványosított formátumban kapja meg a hiba részleteit.

Jellemző HTTP állapotkódok:

- **400 Bad Request:** Validációs, paraméter- vagy formátumhibák esetén.
- **403 Forbidden:** Jogosultsági problémák kezelésére.
- **404 Not Found:** Nem létező erőforrás hivatkozásakor.
- **409 Conflict:** Üzleti logikai ütközések (pl. már foglalt időpont) esetén.
- **415 Unsupported Media Type:** Nem támogatott tartalomtípus küldésekor.
- **500 Internal Server Error:** Váratlan szervertoldali hibákhoz.

A pontosabb diagnózis érdekében a rendszer a validációs hibákhoz részletes **fieldErrors** vagy **parameterErrors** listát is mellékel, amely megjelöli a hiba pontos helyét és okát.

13. ER Diagram



1.10 Kép: Táblák kapcsolatait és kardinalitásait jól bemutatja.

14. Adatbázis dokumentáció

Adatbázis neve: easyarrange

A rendszer adatainak tárolásáért egy **MySQL (MariaDB)** relációs adatbázis-kezelő rendszer felel. A tervezés során törekedtem a redundancia mentességre és a logikai integritás megőrzésére (harmadik normálforma, 3NF).

14.1. Entity-Relationship (ER) Modell és Kapcsolatok

Az adatbázis központi eleme a felhasználók kezelése, a szolgáltatások definiálása és az ezeket összekötő időpontfoglalási logika.

14.1.1 Főbb kapcsolati típusok:

1:N (Egy-a-többhöz): Egy felhasználónak vagy munkatárnak több foglalása is lehet, de egy foglalás pontosan egy ügyfélhez és egy munkatárshoz tartozik.

N:M (Több-a-többhöz): A staff és service táblák között, valamint a staff és shift táblák között több-a-több kapcsolat áll fenn, melyeket kapcsolótáblákkal (staff_service, staff_shift) oldottunk meg.

14.2. Adattáblák részletes leírása

14.2.1. Felhasználókezelés és Szerepkörök

user: Tárolja a rendszer minden felhasználóját (admin, dolgozó, vendég). Biztonsági okokból a jelszavak BCrypt hash-elve kerülnek mentésre.

role: Definiálja a jogosultsági szinteket (ADMIN, CUSTOMER, STAFF).

staff: A user tábla kiterjesztése. Itt tároljuk a dolgozók speciális adatait, mint a bemutatkozó szöveg (bio) és a szakmai megnevezés (title).

14.2.2. Szolgáltatások és Időbeosztás

service: A szalon által kínált szolgáltatások listája (név, ár, időtartam, leírás, kép).

shift: A választható műszakokat és időszavakat tartalmazza napokra lebontva.

calendar_block: Lehetővé teszi a dolgozók számára, hogy manuálisan blokkoljanak időszavakat (pl. szabadság vagy ebédidő), így oda nem érkezhetsz foglalás.

14.2.3. Foglalási rendszer

booking: Az adatbázis "szíve". Összeköti a vendéget, a dolgozót és a szolgáltatást egy adott időintervallummal.

Állapotkezelés: A status mező (Enum) követi a foglalás életútját: BOOKED, CANCELLED, COMPLETED, NO_SHOW.

Integritás: Egy generált mező (active_booking_key) és egy egyedi index gondoskodik arról, hogy ugyanarra az időpontra ne lehessen dupla foglalást leadni, ha az előző még aktív.

14.3. Adatszótár (Fontosabb táblák szerkezete)

Az alábbi táblázat az adatbázis legfontosabb tábláit és azok szerkezetét mutatja be. Tartalmazza az egyes táblák nevét, rövid leírását, elsődleges kulcsait, valamint a köztük lévő kapcsolatokat meghatározó idegen kulcsokat.

A *user* tábla a felhasználók alapadatait tárolja, míg a *staff* tábla az alkalmazottak profilját tartalmazza, és kapcsolatban áll a felhasználókkal. A *service* tábla a rendszerben elérhető szolgáltatásokat írja le. A *booking* tábla az időpontfoglalásokat kezeli, összekapcsolva a felhasználókat, az alkalmazottakat és a szolgáltatásokat. A *staff_shift* pedig a munkatársak és a műszakok közötti kapcsolatot rögzíti.

Tábla neve	Leírás	Elsődleges kulcs	Idegen kulcs(ok)
user	Felhasználói alapadatok	user_id	role_id
staff	Alkalmazottak profilja	staff_id	user_id
booking	Időpontfoglalások	booking_id	staff_id, user_id, service_id
service	Szolgáltatások adatai	service_id	-
staff_shift	Munkatárs-műszak kapcsolat	Összetett	staff_id, shift_id

14.4. SQL Megoldások és optimalizálás

A fejlesztés során több speciális SQL megoldást alkalmaztunk a megbízhatóság érdekében:

Tranzakciókezelés: A foglalások rögzítésekor tranzakciókat használunk, hogy elkerüljük a részleges adatmentést hiba esetén.

Constraint-ek (Megkötések): ON DELETE CASCADE és ON UPDATE CASCADE szabályokat állítottunk be, így, ha egy felhasználó vagy szolgáltatás törlésre kerül, a kapcsolódó rekordok (pl. foglalások) is megfelelően kezelve lesznek, elkerülve az árván maradt adatokat.

Karakterkódolás: Egységesen utf8mb4_hungarian_ci kódolást használtunk a magyar ékezetes karakterek (pl. Aladár, Pedikűr) helyes tárolása és sorbarendezése érdekében.

15. Csapatmunka

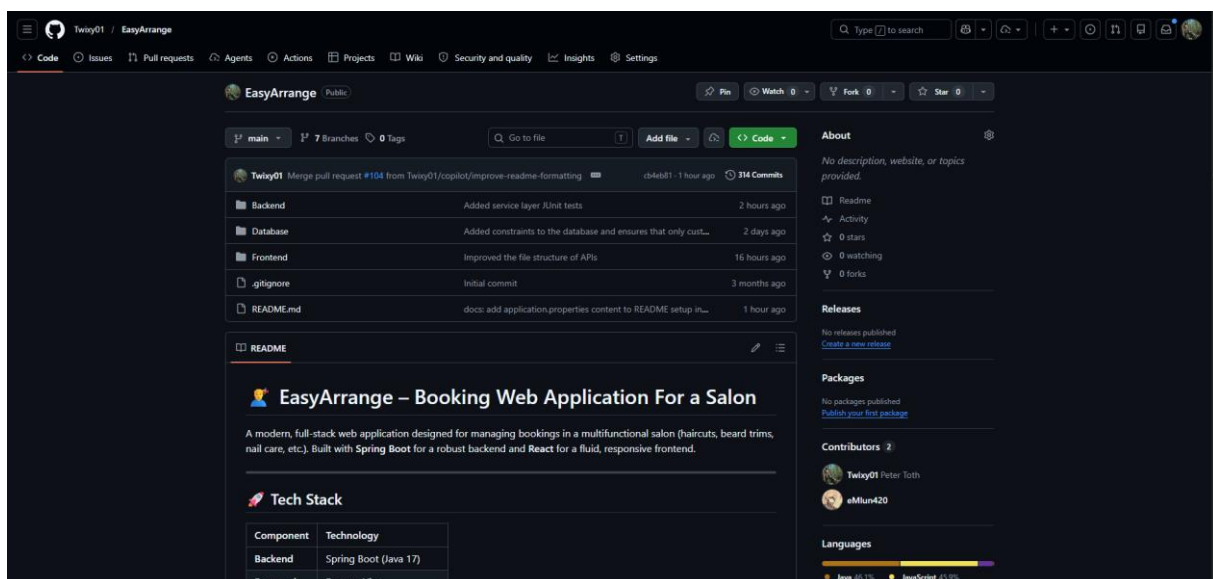
A csapatmunkát ebben a fejezetben részletesen bemutatjuk, mert fontos részét képezi a fejlesztési folyamatnak.

15.1 Verziókezelés

A közös munka során Git verziókezelőt használtunk, a kódot pedig a GitHubon tároltuk, így bármikor vissza tudtuk nézni a korábbi módosításokat, és pontosan láttuk, ki mit változtatott. A teendőket Trello táblán vezettük és kéthetes szakaszokra bontva haladtunk előre.

A fejlesztés során minden új funkciót vagy javítást külön szálon, úgynevezett branch-eken készítettünk el, hogy ne zavarjuk egymás munkáját és megőrizzük a fő programkód épségét. Figyeltünk arra is, hogy ne nyúljunk ugyanazokba a fájlokba egyszerre, így elkerülve a merge conflict-okat.

Mielőtt bármilyen újdonság bekerült volna a végleges kódba, Pull Request-eket küldtünk egymásnak. Ez egyfajta ellenőrző pontként szolgált, ahol alaposan átnéztük a másik kódját, és ha hibát találtunk vagy jobb megoldást javasoltunk, azt jeleztük a másiknak. Csak a közös jóváhagyás után adtuk hozzá a változtatásokat a fő ághoz, ami garantálta, hogy a rendszer folyamatosan az elképzelt módon működjön.



1.11 Kép: EasyArrange GitHub repository fő oldala

A projekt tervezése és elkezdése előtt úgy döntöttünk, hogy megosztjuk a munkát. Ezért közösen, fele-fele arányban dolgoztunk a projektünk backend, frontend és az adatbázis részén is. GitHubon belül külön brancheken haladtunk a feladatainkkal, amiket összeírtunk 2 hetes sprintekre leosztva, melyeket a Trello webalkalmazásban követtünk. Fontos kiemelnem, hogy sosem dolgoztunk ugyanazon feladatokon és brancheken a merge conflictok elkerülése érdekében.

16. Fejlesztési módszertan

16.1 Módszertani keret

A projekt iteratív, inkrementális megközelítést követ: kis lépésekben történő fejlesztés, folyamatos visszajelzésalapú javítás, gyors hibajavítás és stabilizálás minden kör végén.

16.1.1 Agilis módszertan: gyors visszacsatolások és folyamatos stabilizálás

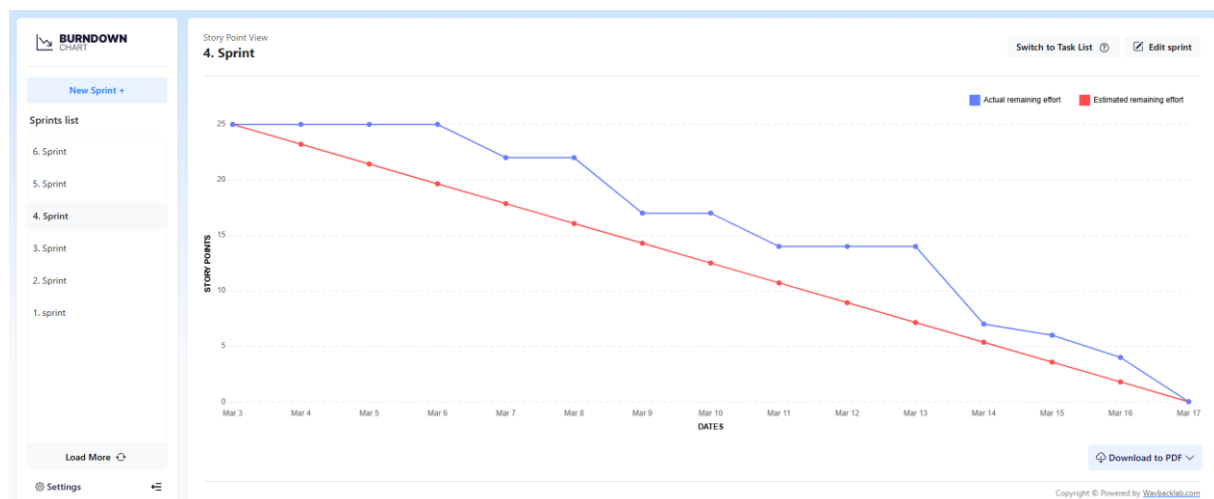
A projektben az agilis ciklusokat minden fejlesztési lépés után gyors ellenőrzés követi (build + célzott tesztfuttatás).

Ez biztosítja, hogy a hibák korán felismerhetők és javíthatók legyenek, miközben a kódbázis végig futtatható és konzisztens marad.

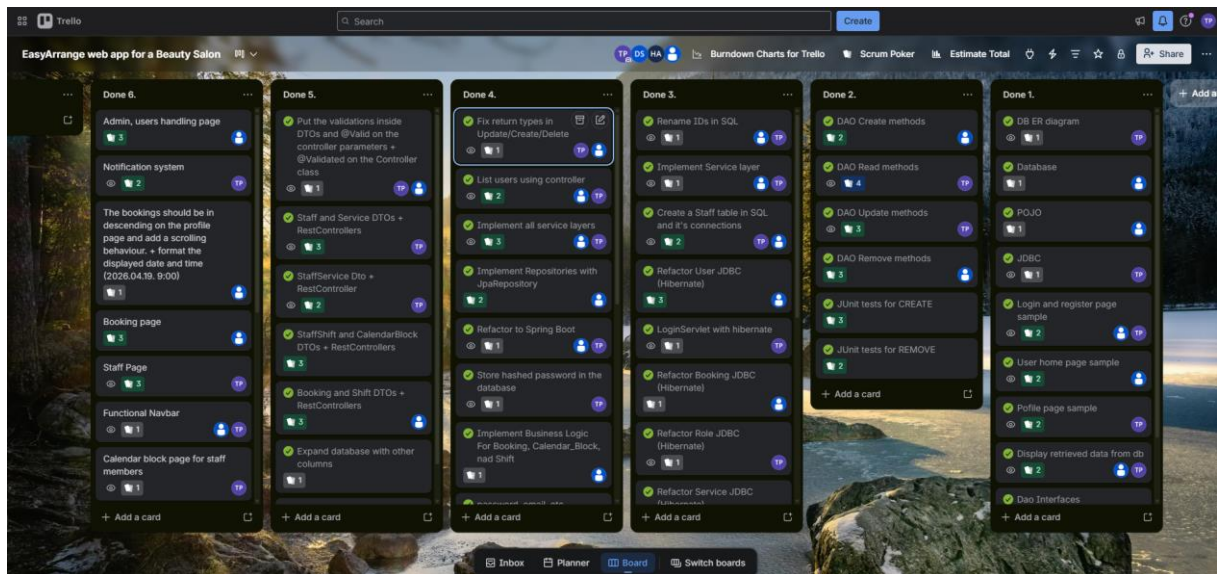
A megközelítés eredménye: kisebb regressziós kockázat, gyorsabb hibajavítás, stabilabb kiadások.

16.1.2 Két hetes sprintek Trello használatával

A projekt munkaszervezése 2 hetes sprintekben történt, Trello alapú feladatkezeléssel (backlog, folyamatban lévő feladatok, kész elemek). A sprint előrehaladását burndown chart segítségével követtük, ami láthatóvá tette a tervezett és tényleges teljesítést, így támogatva a korai csúszásészlelést és a gyors újratervezést.



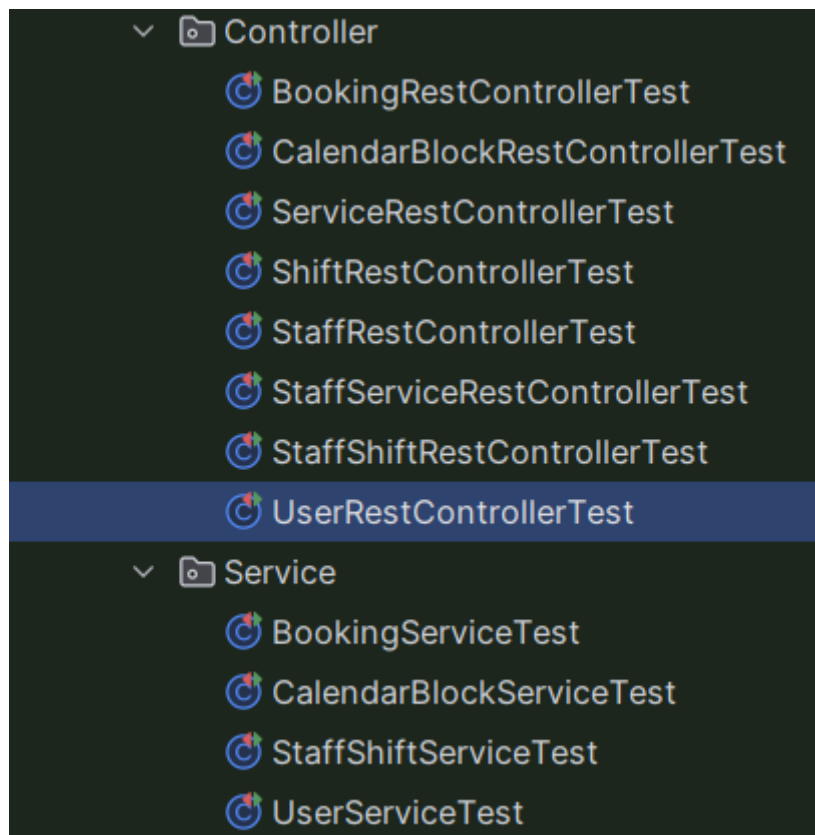
1.12 Kép: Trello BurnDown chart (2 hetes sprint példa)



1.13 Kép: Trelloban a sprintek elkészült issue-i láthatóak.

16.2 Tesztelési stratégia

A tesztelési stratégia elsődleges fókuszja a **backend controller és service rétegek** egységtesztelése, mivel ezek a komponensek felelősek az üzleti logika végrehajtásáért és az API végpontok helyes működéséért. A cél az egyes rétegek izolált validálása és a komponensek közötti együttműködés ellenőrzése előre definiált elvárások szerint.



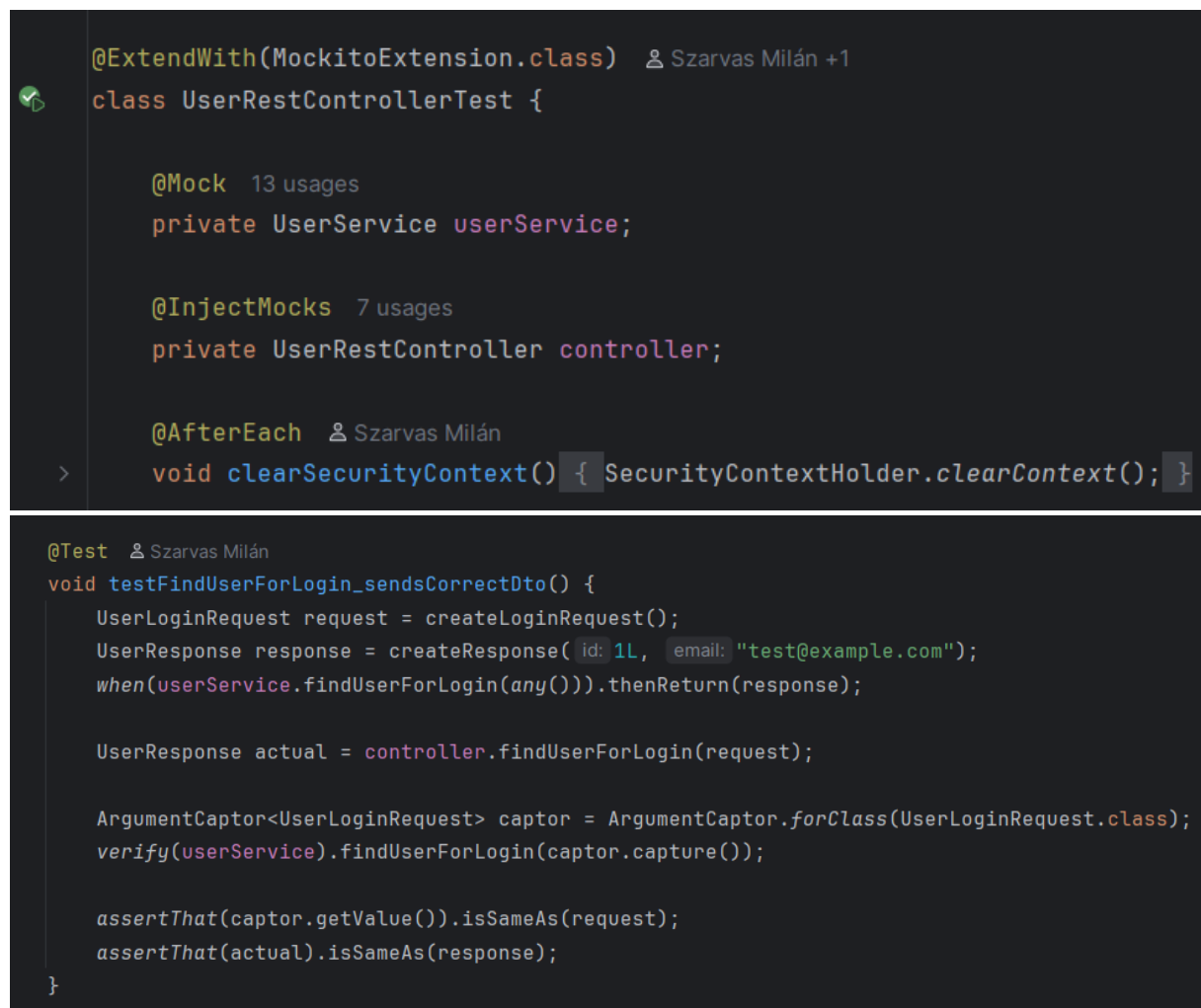
Alkalmazott technológiák és módszertan

A tesztelés a **JUnit 5** és **Mockito** keretrendszerekre épül (`@ExtendWith`, `@Mock`, `@InjectMocks`). A megbízhatóság érdekében minden teszt:

Izolált: Az adatbázistól függetlenül, mockolt függőségekkel működik.

Alapos: Lefedi a "happy-path" forgatókönyveket és a hibakezelési logikát is.

Precíz: Az interakciókat `verify()` hívásokkal, a paraméterátadást pedig `ArgumentCaptor` használatával validálja.



```
@ExtendWith(MockitoExtension.class)
class UserRestControllerTest {

    @Mock
    private UserService userService;

    @InjectMocks
    private UserRestController controller;

    @AfterEach
    void clearSecurityContext() { SecurityContextHolder.clearContext(); }

    @Test
    void testFindUserForLogin_sendsCorrectDto() {
        UserLoginRequest request = createLoginRequest();
        UserResponse response = createResponse(id: 1L, email: "test@example.com");
        when(userService.findUserForLogin(any())).thenReturn(response);

        UserResponse actual = controller.findUserForLogin(request);

        ArgumentCaptor<UserLoginRequest> captor = ArgumentCaptor.forClass(UserLoginRequest.class);
        verify(userService).findUserForLogin(captor.capture());

        assertThat(captor.getValue()).isSameAs(request);
        assertThat(actual).isSameAs(response);
    }
}
```

1.14 Kép: A tesztjeink felépítését bemutató képet láthatunk

16.2.1 Tesztcsomagok és lefedettség

A tesztcsomag az alkalmazás kritikus üzleti logikáját az alábbiak szerint fedi le:

BookingServiceTest (5 teszt): Az időpontfoglalás alapkövei, beleértve az ütközés-érzékelést, a szabad időpontok kalkulációját és a lemondási feltételek ellenőrzését.

CalendarBlockServiceTest (7 teszt): A naptárblokkolások kezelése, különös tekintettel a szerepkör-ellenőrzésre, a duplikátumok szűrésére és a módosítási műveletekre.

StaffShiftServiceTest (7 teszt): A munkatársi műszakok életciklusa, a műszakcserék validációja, valamint az időtartam- és napalapú korlátozások ellenőrzése.

UserServiceTest (5 teszt): A felhasználói adatok biztonsága, a jelszókódolás folyamata, a bejelentkezési logika és a profilfrissítések validálása.

Ez a megközelítés garantálja, hogy a rendszer üzleti szabályai technikai szinten is konzisztensek maradjanak minden módosítás után.

16.3 Minőségbiztosítási gyakorlat

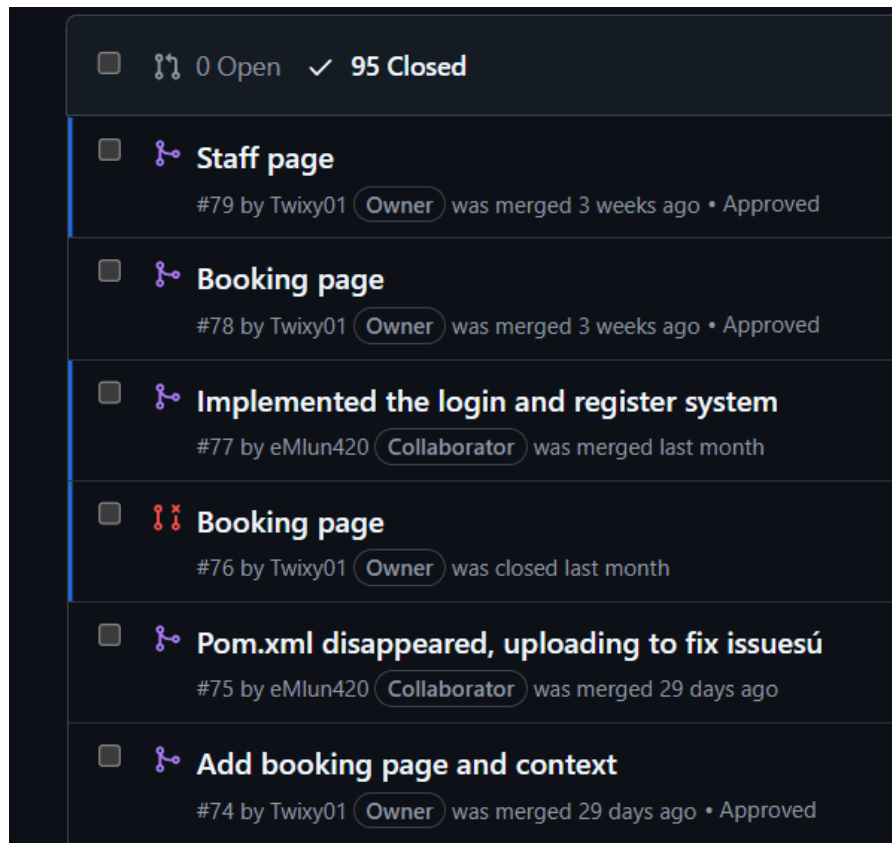
A beszélgetés alapján a QA folyamat fő elemei: - strukturális integritás ellenőrzése (package/import/class sorrend), - fordítási hibák gyors lokalizálása és javítása, - konzisztencia a tesztosztályok között, - figyelmeztetések csökkentése (pl. nem használt importok eltávolítása). Ez különösen fontos volt a korábban előforduló fájlsérülési/reorder problémák kezelésében.

16.4 Build és futtatási folyamat

A projekt Maven alapú, lokális validáció parancssoros és IDE környezetben történik (IntelliJ IDEA): - célzott tesztfuttatás csomagszinten/pl. controller és service layer tesztekre - hibák esetén gyors patchelés, majd újrafuttatás.

16.5 Verziókezelés és együttműködés

Git alapú munkavégzés történik, központi távoli tárral (origin), main ágon. A gyakorlatban ez: változtatások nyomon követését, reprodukálható hibajavítást és dokumentálható fejlesztési lépéseket biztosít.



1.15 Kép: Githubon az origin main ágra érkező pull request-ek egy kis része.

17. Jövőbeli fejlesztési lehetőségek

Egy ehhez hasonló projektet mindig lehet finomítani és tovább fejleszteni. Bemutatjuk az alkalmazás lehetséges továbbfejlesztési irányait és bővítési lehetőségeit. A javaslatok magukba foglalják új funkciók bevezetését, technológiai frissítéseket, valamint a felhasználói élmény és a rendszereffektivitás javítását célzó fejlesztéseket. A cél, hogy az alkalmazás folyamatosan alkalmazkodjon a piaci igényekhez és technológiai trendekhez.

1. Megerősítő email kiküldése regisztrációkor.
2. Értékelés küldésének a lehetősége a szolgáltatónak, a szolgáltatás befejezését követően.
3. Emlékeztető email küldése a szolgáltatónak és az ügyfélnek is az időpont közeledéséről.
4. Szolgáltatónak név szerinti keresés lehetőség a felhasználók – és a szolgáltatások kezelése oldalon.
5. Email értesítés küldése az ügyfélnek a foglalás állapotának a megváltoztatásáról (lemondott, nem jelent meg)
6. Fizetési rendszer megvalósítása -> előleg (30%) befizetésének a szükségessége foglalás esetén.
7. Bevételek nyilvántartása és statisztikai adatok megjelenítése (Pl.: heti / havi / éves bevétel)
8. JWT alapú hitelesítés
9. SecurityConfig jogosultságellenőrzés megvalósítása backend oldalon is, hogy jogosulatlan felhasználó ne tudjon érzékeny adatokat lekérni és kezelni végpontok és módszaraik használatával.

18. Projekt elérése

Github repository: <https://github.com/Twixy01/EasyArrange>

Felhasználói útmutató: <https://petriklajosszki->

my.sharepoint.com/:w/g/personal/toth_peter_petrik_hu/IQD11ZtNwiwuSrx08cqNENRIAWLdjSjEe2JVB64rVZwTsRA?e=kNfckZ vagy megtalálható githubon a **Dokumentáció mappában**.

19. Források

Az alábbi forrásokat használtuk segítségként a projekt elkészítéséhez.

<https://www.geeksforgeeks.org/software-engineering/software-engineering-agile-software-development/>

<https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/>

<https://edu.nixs.com/>

<https://www.youtube.com/playlist?list=PLsyebzWxl7qbKoSgR5ub6joll8-ocxCF>

<https://www.youtube.com/watch?v=KkxPtimqaew>

<https://react.dev/>

<https://www.geeksforgeeks.org/springboot/spring-boot-jparepository-with-example/>

<https://spring.io/projects/spring-boot>

<https://images.unsplash.com/>